Name: Madhu M M

Enterprise Data Governance & Ingestion Framework for Insurance Domain

End-to-end case study: SQL schemas → ADF → data Bricks

This document shows clear, step-by-step instructions from creating SQL schemas to running Databricks notebooks from Azure Data Factory.

Overview

This guide walks through:

1. create database + tables in SQL server (lowercase) and insert sample rows.
2. push raw files to azure data lake storage under raw/bronze/
3. build an ADF pipeline that reads a list of table names, loops and copies each table to blob
4. run data Bricks notebooks from ADF to validate, mask and write silver (delta)
5. transform silver into gold (business table)
6. monitoring, artifacts and demo checklist

Prerequisites

- azure subscription with storage account and data Bricks workspace
- AAD user with permission to create resources.
- azure data factory instance
- SQL server (or azure SQL) accessible from ADF or via linked service

• data Bricks token (for linking to ADF)

Project Flow:

1. Create database of Insurance

- **Purpose**

This section outlines the foundational setup of the source SQL database schema and introduces a metadata-driven approach to manage dynamic data ingestion through Azure Data Factory (ADF). It forms the first step in building a scalable and reusable end-to-end data pipeline from SQL to Azure Data Lake, enabling efficient data movement and transformation in a modern data platform architecture.

1. SQL Tables Creation

I created a comprehensive insurance-related relational schema named PolicyDB that reflects typical business domains including policy management, customer information, payments, claims, and supporting entities.

Key Tables:

- **PolicyHolder** – Stores customer personal details
- **Policy** – Main policy information linked to policyholders and statuses
- **PolicyStatus** – Lookup table for policy lifecycle statuses
- **Coverage** – Details of insurance coverage per policy
- **Premium** – Premium payment schedules
- **Claims** – Insurance claim records
- **Beneficiaries** – Beneficiaries linked to policies
- **Agents** – Sales or support agents
- **PolicyDocuments** – Related policy documents
- **Payments** – Actual payments received

Each table is well-normalized with appropriate foreign key constraints to preserve referential integrity.

For example:

Policy.PolicyHolderID → references PolicyHolder.PolicyHolderID
Policy.StatusID → references PolicyStatus.StatusID

2. Sample Data Insertion

Each table is populated with realistic and meaningful sample data:
20 policyholders, each with a corresponding policy Coverages, claims, premiums, payments, and documents are linked to these policies Diverse combinations of policy types, claim statuses, and payment methods. This synthetic data enables thorough testing and simulation of downstream processes such as:

- Data ingestion pipelines
- Transformation logic in Databricks
- Data masking and validation
- Analytics on claims, premiums, and policyholder behavior

3. Metadata Table – MetadataDataFlowConfig

A key component in enabling dynamic ingestion via ADF is the MetadataDataFlowConfig table. This table drives the orchestration logic in the ADF pipeline by storing metadata for each table to be copied.
Columns:

- **TableName** – Logical name of the source table
- **SourceQuery** – SQL query to extract the data
- **SinkPath** – Target path in Azure Data Lake (under /raw/bronze/)

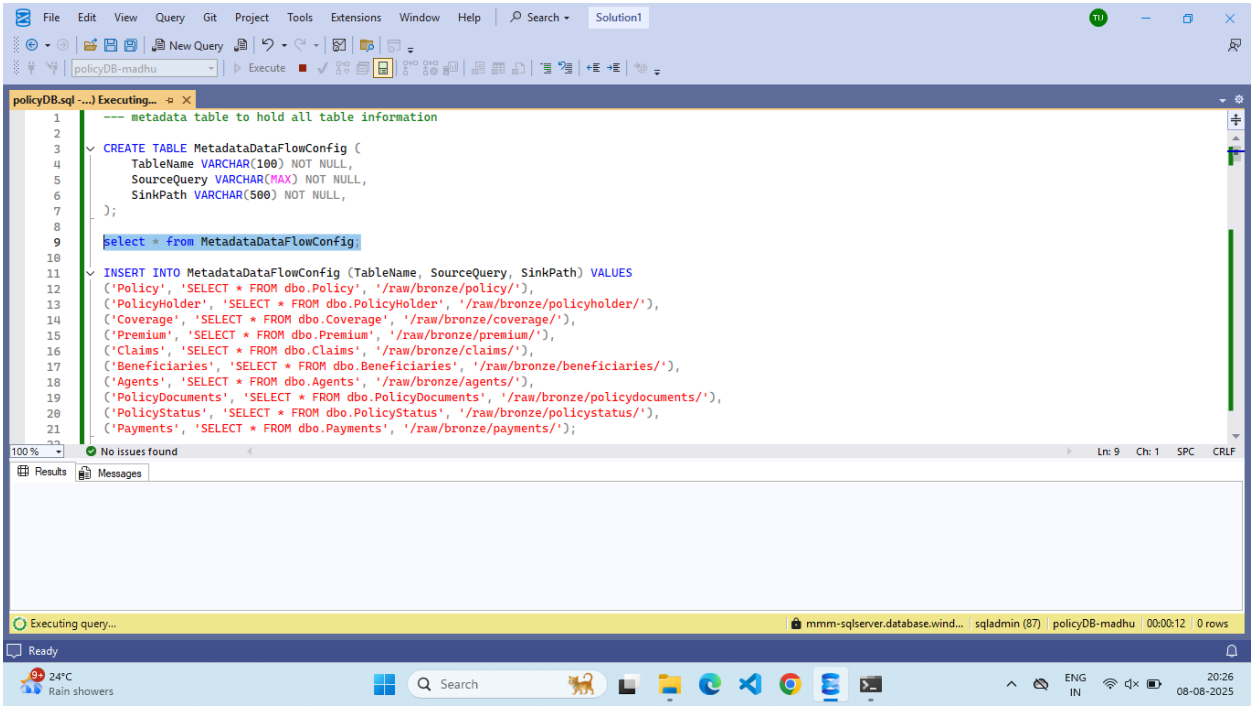
Example entries:

('Policy', 'SELECT * FROM dbo.Policy', '/raw/bronze/policy/')
('Claims', 'SELECT * FROM dbo.Claims', '/raw/bronze/claims/')
('Payments', 'SELECT * FROM dbo.Payments', '/raw/bronze/payments/')

This design pattern allows the ADF pipeline to:

- Dynamically loop over multiple tables
- Avoid hardcoding table names or copy logic
- Easily onboard new tables by inserting a new row in the metadata table

Snapshots of SQL Server Database:



Note: This is the only metadata Table. Under these all 10 tables queries and data are present. I have attached full database queries in the case study folder.

2. Azure Data Factory (ADF) Operation

1. Extract Data from SQL Server:

- Use Lookup to read table metadata from MetadataDataFlowConfig
- Query each source table dynamically using metadata
- Parameterize SQL source querie
- **Source Dataset (SQL Server)**

  Type: SQL Server

  Linked Service: Connected to your SQL Server or Azure SQL instance.

  Table or Query: Dynamic query provided via @item().SourceQuery from the Lookup.

  Schema: Based on metadata (MetadataDataFlowConfig).

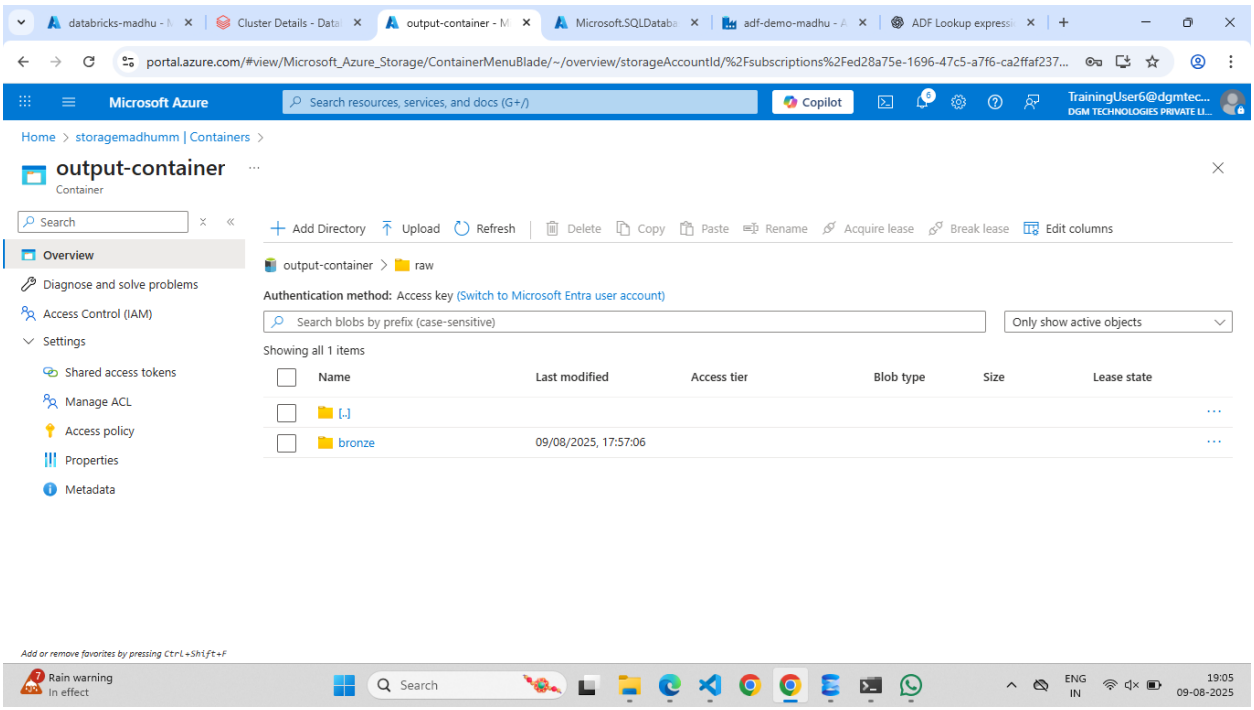
- **Sink Dataset (Azure Data Lake / Blob Storage)**

Type: Azure Data Lake Storage Gen2

Linked Service: Connected to the ADLS account.

Folder Path: Dynamic, retrieved from @item().SinkPath (e.g., /raw/bronze/policy/)

File Format: Parquet.



3. ADF Pipeline – Data Extraction & Transformation Flow:

This pipeline is designed to **dynamically extract data from multiple SQL Server tables** using metadata and store it in **Azure Data Lake Storage (ADLS)**. It also triggers **Databricks notebooks** for further transformation.

Pipeline Components:

1. Lookup Activity – Lookup1

Purpose: Reads table metadata from MetadataDataFlowConfig table.

What it does:

Retrieves the list of tables with TableName, SourceQuery, and SinkPath.

Output: Array of table configurations passed to ForEach loop.

2. ForEach Activity – ForEach1

Purpose: Iterates through each row from the Lookup activity (i.e., each table).

What it does: Executes the copy and transformation steps for each table.

Items: Takes input from @activity('Lookup1').output.value

Inside the ForEach loop:

a. Copy Data Activity – Copy data1

Purpose: Extracts data from SQL and loads it into ADLS.

Source:

- Azure SQL Database or SQL Server
- Uses dynamic query (@item().SourceQuery)

Sink:

- ADLS Gen2 or Blob
- Writes to dynamic path (@item().SinkPath)

File Format: Parquet

3. Databricks Notebook Activity – Notebook1

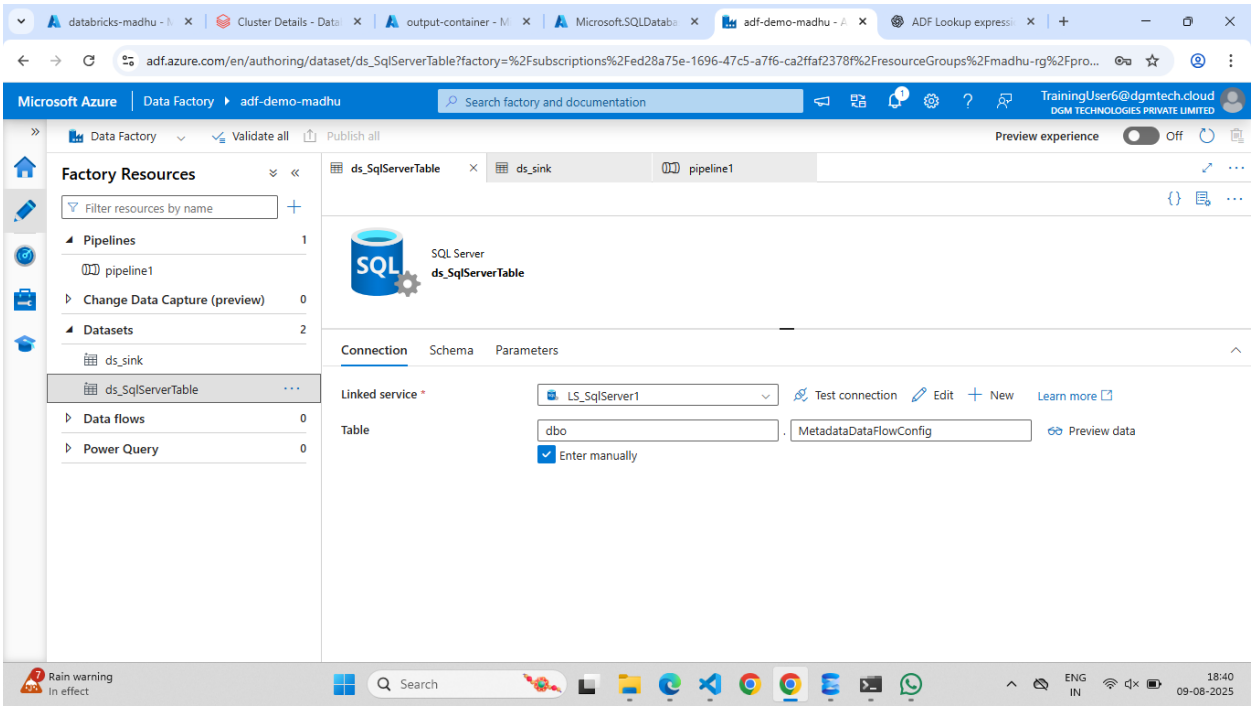
Purpose: Triggers a Databricks notebook from ADF

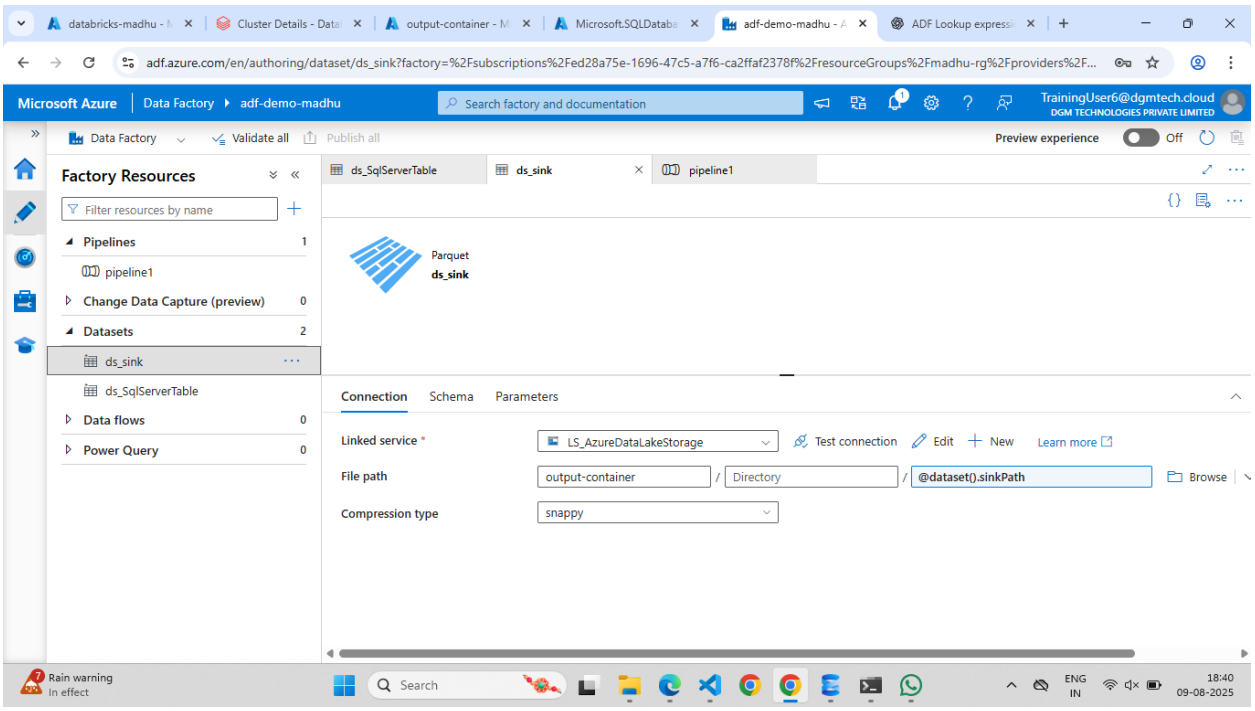
Functionality:

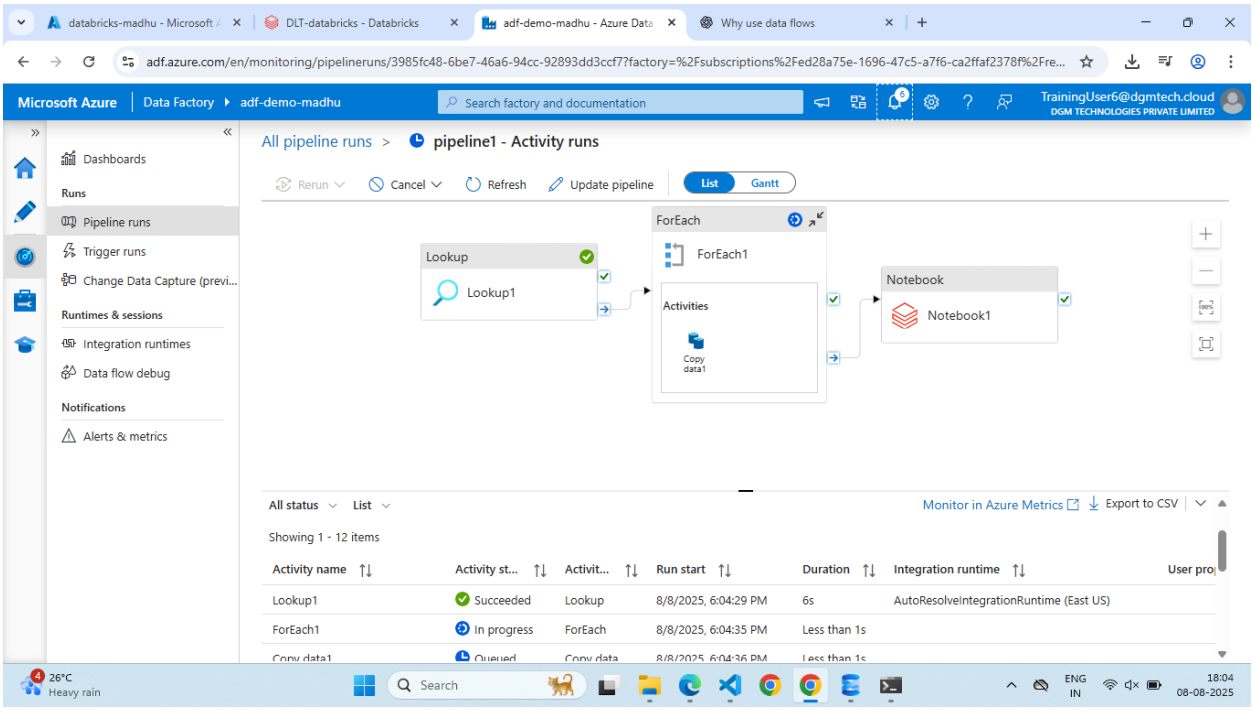
- Reads the raw file from ADLS

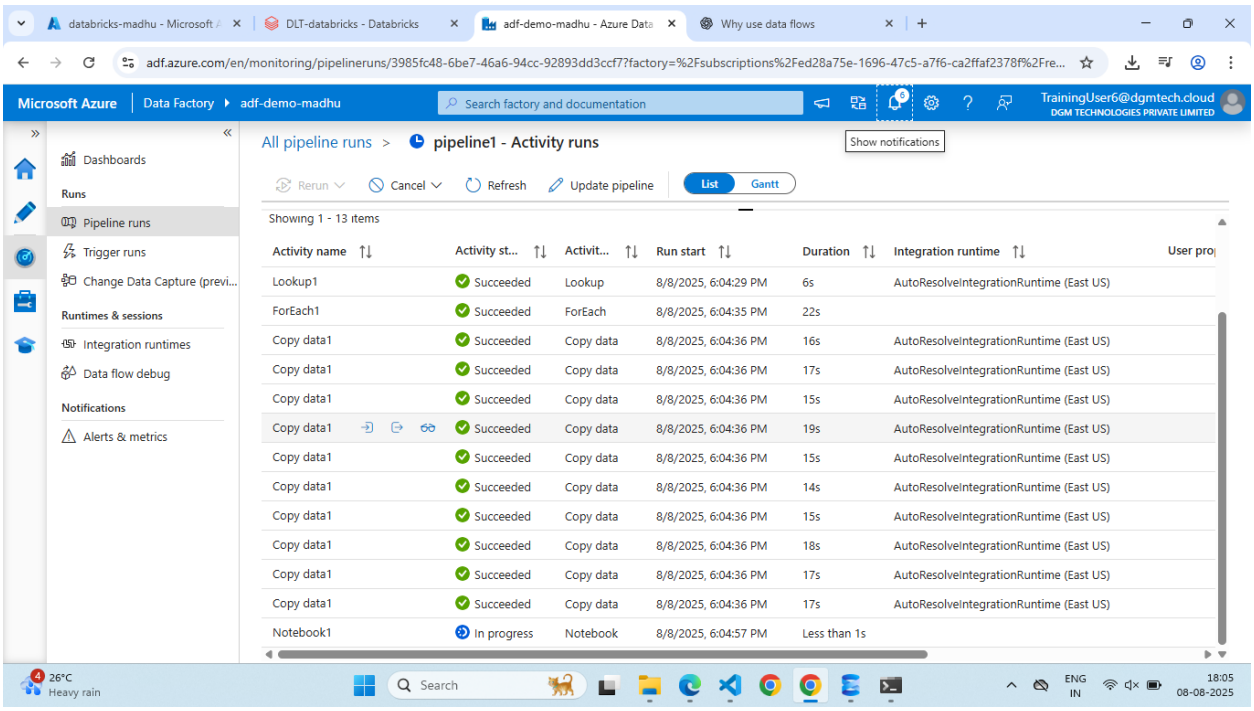
- Applies validation, masking, or transformation logic
- Writes the cleaned data to Silver layer (Delta format)

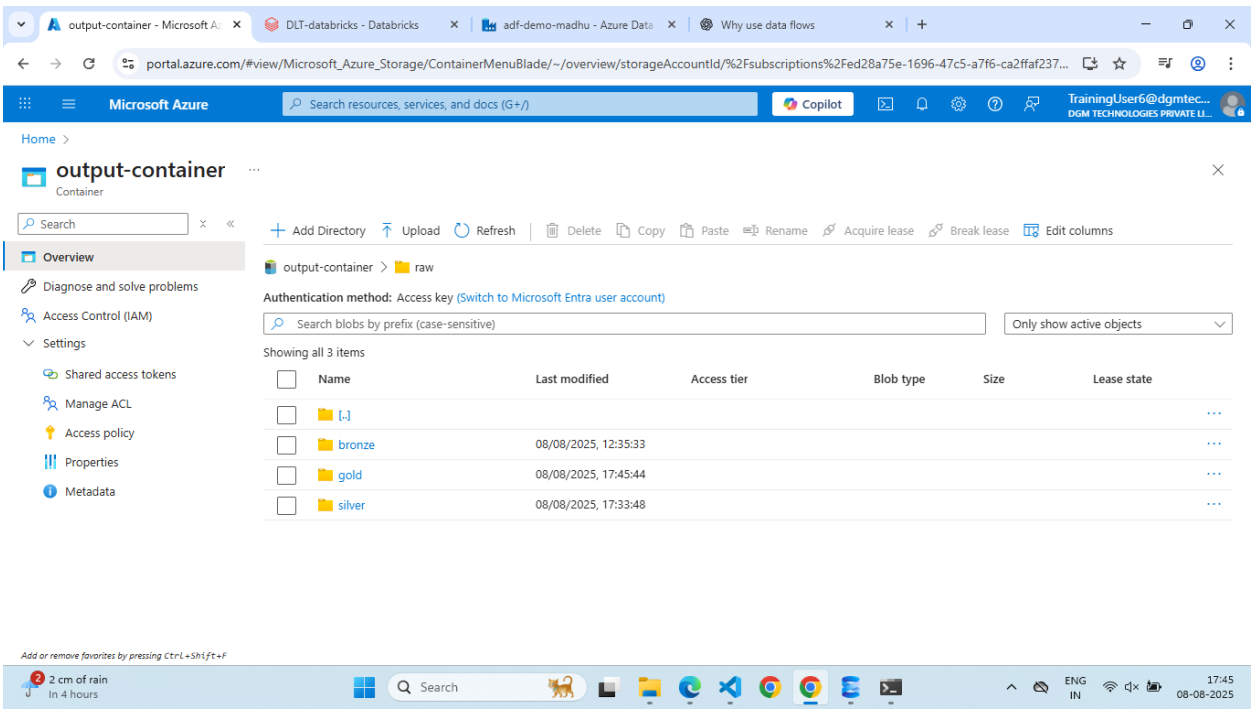
Snapshots:




Microsoft Azure
Data Factory
adf-demo-madhu
Search factory and documentation
TrainingUser6@dgmtech.cloud
DGM TECHNOLOGIES PRIVATE LIMITED
Validate all
Publish all
Preview experience
Off
Factory Resources
Filter resources by name
Pipelines 1
pipeline1
Change Data Capture (preview) 0
Datasets 2
ds_sink
ds_SqlServerTable
Data flows 0
Power Query 0
ds_SqlServerTable
ds_sink
pipeline1
Parquet
ds_sink
Connection
Schema
Parameters
Linked service *
LS_AzureDataLakeStorage
Test connection
Edit
New
Learn more
File path
output-container
Directory
@dataset().sinkPath
Browse
Compression type
snappy
Rain warning
In effect
Search
ENG
IN
18:40
09-08-2025


Microsoft Azure
Data Factory
adf-demo-madhu
Search factory and documentation
TrainingUser6@dgmtech.cloud
DGM TECHNOLOGIES PRIVATE LIMITED
Dashboards
Runs
Pipeline runs
Trigger runs
Change Data Capture (previ...
Runtimes & sessions
Integration runtimes
Data flow debug
Notifications
Alerts & metrics
All pipeline runs > pipeline1 - Activity runs
Rerun
Cancel
Refresh
Update pipeline
List
Gantt
Lookup
Lookup1
ForEach
ForEach1
Activities
Copy data1
Notebook
Notebook1
All status
List
Monitor in Azure Metrics
Export to CSV
Showing 1 - 12 items
Activity name
Activity st...
Activit...
Run start
Duration
Integration runtime
User pro
Lookup1
Succeeded
Lookup
8/8/2025, 6:04:29 PM
6s
AutoResolveIntegrationRuntime (East US)
ForEach1
In progress
ForEach
8/8/2025, 6:04:35 PM
Less than 1s
Copy data1
Queued
Copy data
8/8/2025, 6:04:36 PM
Less than 1s
26°C
Heavy rain
Search
ENG
IN
18:04
08-08-2025





Note: I have attached **Notebook** and and **queries** in zip folder.